

SQL: Les fonctions de fenêtrage

les fonctions SQL window permettent d'afficher *toutes les lignes de données et leurs valeurs agrégées en même temps*. Cela ressemble un peu à GROUP BY sous stéroïdes, n'est-ce pas ? Mais ce n'est pas tout. Comme les fonctions de fenêtre effectuent divers calculs sur l'ensemble des lignes, *elles permettent également de diviser les ensembles de données en sous-ensembles*. Elles vous aident à classer facilement les données, à les agréger, à calculer les différences entre différentes périodes (par exemple, augmentation/diminution des bénéfices entre deux mois/trimestres/années) et à trouver des sommes cumulées, des totaux courants, des moyennes mobiles, etc.

Syntaxe:

```
fonction_window() OVER (  
  [PARTITION BY colonnes]  
  [ORDER BY colonnes]  
  [frame_clause]  
)
```

- *fonction_window()* est la fonction appliquée (comme SUM, AVG, ROW_NUMBER, etc.)
- *PARTITION BY* divise les résultats en partitions sur lesquelles la fonction est appliquée
- *ORDER BY* définit l'ordre des lignes dans chaque partition
- *frame_clause* précise l'ensemble des lignes constituant la fenêtre pour chaque ligne

Préparation des données d'exemple

-- Table des ventes par région

CREATE TABLE ventes_regionales (

id INTEGER PRIMARY KEY,

region TEXT,

annee INTEGER,

trimestre INTEGER,

montant REAL

);

-- Insertion de données d'exemple

INSERT INTO ventes_regionales (region, annee, trimestre, montant) VALUES

('Nord', 2022, 1, 45000),

('Nord', 2022, 2, 52000),

('Nord', 2022, 3, 49000),

('Nord', 2022, 4, 58000),

('Sud', 2022, 1, 38000),

('Sud', 2022, 2, 40000),

('Sud', 2022, 3, 43000),

('Sud', 2022, 4, 47000),

('Est', 2022, 1, 32000),

('Est', 2022, 2, 35000),

('Est', 2022, 3, 37000),

('Est', 2022, 4, 41000),

('Ouest', 2022, 1, 51000),

('Ouest', 2022, 2, 53000),

('Ouest', 2022, 3, 56000),

('Ouest', 2022, 4, 59000);

-- Table des employés et leurs performances

CREATE TABLE performances_employes (

id INTEGER PRIMARY KEY,

nom TEXT,

departement TEXT,

score_performance INTEGER,

salaire REAL

);

-- Insertion de données d'exemple

```
INSERT INTO performances_employees (nom, departement, score_performance, salaire) VALUES  
('Dupont', 'Ventes', 85, 45000),  
('Martin', 'Ventes', 92, 48000),  
('Durand', 'Ventes', 78, 42000),  
('Petit', 'Ventes', 95, 52000),  
('Leroy', 'Marketing', 88, 46000),  
('Moreau', 'Marketing', 79, 43000),  
('Dubois', 'Marketing', 91, 49000),  
('Simon', 'Marketing', 84, 45000),  
('Laurent', 'IT', 97, 56000),  
('Michel', 'IT', 94, 54000),  
('Lefebvre', 'IT', 89, 51000),  
('Garcia', 'IT', 86, 50000);
```

Fonctions de classement (Ranking Functions) ROW NUMBER, RANK et DENSE RANK

Ces fonctions attribuent des numéros ou des rangs aux lignes dans une partition ordonnée:

-- *Classement des employés par score de performance*

```
SELECT
```

```
  nom,
```

```
  departement,
```

```
  score_performance,
```

```
  ROW_NUMBER() OVER (ORDER BY score_performance DESC) AS position,
```

```
  RANK() OVER (ORDER BY score_performance DESC) AS rang,
```

```
  DENSE_RANK() OVER (ORDER BY score_performance DESC) AS rang_dense
```

```
FROM performances_employes;
```

-- Classement des employés par département

SELECT

nom,

departement,

score_performance,

ROW_NUMBER() OVER (PARTITION BY departement ORDER BY score_performance
DESC) AS position_dept,

RANK() OVER (PARTITION BY departement ORDER BY score_performance DESC) AS
rang_dept,

DENSE_RANK() OVER (PARTITION BY departement ORDER BY score_performance
DESC) AS rang_dense_dept

FROM performances_employes;

Différences entre ces fonctions :

ROW_NUMBER() : Attribue un numéro unique à chaque ligne (1, 2, 3, 4, ...)

RANK() : Attribue le même rang aux ex-aequo, puis saute les rangs suivants (1, 1, 3, 4, ...)

DENSE_RANK() : Attribue le même rang aux ex-aequo, sans sauter de rangs (1, 1, 2, 3, ...)

NTILE

La fonction NTILE divise les lignes en un nombre spécifié de groupes aussi égaux que possible :

-- Diviser les employés en 3 groupes selon leur performance

```
SELECT
```

```
  nom,
```

```
  score_performance,
```

```
  NTILE(3) OVER (ORDER BY score_performance DESC) AS tiers
```

```
FROM performances_employes;
```

Cette requête classe les employés en trois tiers (1 = meilleurs performances, 3 = performances plus faibles).

Fonctions d'agrégation dans les fenêtres

Les fonctions d'agrégation classiques peuvent être utilisées comme fonctions de fenêtrage:

-- Calcul de la somme cumulée des ventes par région

```
SELECT
```

```
  region,
```

```
  trimestre,
```

```
  montant,
```

```
  SUM(montant) OVER (PARTITION BY region ORDER BY trimestre) AS cumul_ventes
```

```
FROM ventes_regionales
```

```
ORDER BY region, trimestre;
```

-- Calcul de la moyenne mobile des ventes sur 2 trimestres

SELECT

region,

trimestre,

montant,

AVG(montant) OVER (PARTITION BY region ORDER BY trimestre

ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) AS
moyenne_mobile_2_trim

FROM ventes_regionales

ORDER BY region, trimestre;

-- Pourcentage du total par région

SELECT

region,

trimestre,

montant,

ROUND(montant * 100.0 / SUM(montant) OVER (PARTITION BY region), 2) AS
pourcentage_region

FROM ventes_regionales

ORDER BY region, trimestre;

Fonctions de navigation (Offset Functions)

Les fonctions LAG et LEAD permettent d'accéder aux valeurs des lignes précédentes ou suivantes:

-- Comparer les ventes au trimestre précédent

```
SELECT region, trimestre, montant,  
       LAG(montant) OVER (PARTITION BY region ORDER BY trimestre) AS montant_trim_precedent,  
       montant - LAG(montant) OVER (PARTITION BY region ORDER BY trimestre) AS difference,  
       CASE  
         WHEN LAG(montant) OVER (PARTITION BY region ORDER BY trimestre) IS NULL THEN NULL  
         ELSE ROUND((montant - LAG(montant) OVER (PARTITION BY region ORDER BY trimestre)) * 100.0 /  
                    LAG(montant) OVER (PARTITION BY region ORDER BY trimestre), 2)  
       END AS pourcentage_evolution  
FROM ventes_regionales  
ORDER BY region, trimestre;
```

-- Comparer avec le trimestre suivant

SELECT

region,

trimestre,

montant,

LEAD(montant) OVER (PARTITION BY region ORDER BY trimestre) AS
montant_trim_suivant

FROM ventes_regionales

ORDER BY region, trimestre;

Les fonctions FIRST_VALUE et LAST_VALUE permettent d'accéder à la première ou dernière valeur de la fenêtre :

-- Écart par rapport à la meilleure performance du département

```
SELECT nom, departement, score_performance,  
       FIRST_VALUE(score_performance) OVER (PARTITION BY departement ORDER BY  
score_performance DESC  
                                           RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED  
FOLLOWING) AS meilleur_score,  
       score_performance - FIRST_VALUE(score_performance) OVER (PARTITION BY departement  
ORDER BY score_performance DESC  
                                           RANGE BETWEEN UNBOUNDED PRECEDING AND  
UNBOUNDED FOLLOWING) AS ecart_meilleur  
FROM performances_employes  
ORDER BY departement, score_performance DESC;
```

Frame Clause (définition de la fenêtre)

La clause de frame vous permet de définir précisément quelles lignes constituent la fenêtre pour chaque ligne :

-- Différentes utilisations de frame clause

```
SELECT
    region,
    trimestre,
    montant,
    SUM(montant) OVER (PARTITION BY region) AS total_region,
    SUM(montant) OVER (PARTITION BY region ORDER BY trimestre) AS cumul_jusqua_maintenant,
    SUM(montant) OVER (PARTITION BY region ORDER BY trimestre
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS somme_mobile_3_trim,
    AVG(montant) OVER (PARTITION BY region ORDER BY trimestre
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS moyenne_jusqua_maintenant
FROM ventes_regionales
ORDER BY region, trimestre;
```

Les options de frame clause les plus courantes sont :

- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW : Du début jusqu'à la ligne actuelle
- ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING : De la ligne actuelle jusqu'à la fin
- ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING : Toutes les lignes de la partition
- ROWS BETWEEN n PRECEDING AND m FOLLOWING : n lignes avant et m lignes après la ligne actuelle

Applications pratiques

Analyse de tendance des ventes

-- Analyse complète des tendances de ventes par région

SELECT

region,

trimestre,

montant,

ROUND(AVG(montant) OVER (PARTITION BY region), 2) AS moyenne_region,

ROUND(montant - AVG(montant) OVER (PARTITION BY region), 2) AS ecart_moyenne,

RANK() OVER (PARTITION BY region ORDER BY montant DESC) AS rang_dans_region,

ROUND(montant * 100.0 / SUM(montant) OVER (PARTITION BY region), 2) AS pourcentage_du_total,

ROUND(montant * 100.0 / SUM(montant) OVER (), 2) AS pourcentage_total_global,

CASE

WHEN LAG(montant) OVER (PARTITION BY region ORDER BY trimestre) IS NULL THEN NULL

ELSE ROUND((montant - LAG(montant) OVER (PARTITION BY region ORDER BY trimestre)) * 100.0 /

LAG(montant) OVER (PARTITION BY region ORDER BY trimestre), 2)

END AS croissance_vs_trim_precedent

FROM ventes_regionales

ORDER BY region, trimestre;

Analyse des salaires

-- Analyse des salaires par rapport à la moyenne du département

SELECT

nom,

departement,

salaire,

ROUND(AVG(salaire) OVER (PARTITION BY departement), 2) AS salaire_moyen_dept,

ROUND(salaire - AVG(salaire) OVER (PARTITION BY departement), 2) AS ecart_moyenne_dept,

ROUND(salaire * 100.0 / AVG(salaire) OVER (PARTITION BY departement) - 100, 2) AS
pct_vs_moyenne_dept,

ROUND(salaire * 100.0 / AVG(salaire) OVER () - 100, 2) AS pct_vs_moyenne_globale,

PERCENT_RANK() OVER (PARTITION BY departement ORDER BY salaire) AS percentile_dept

FROM performances_employes

ORDER BY departement, salaire DESC;

Identification des quartiles

-- Déterminer les quartiles de performance

```
SELECT
  nom,
  departement,
  score_performance,
  NTILE(4) OVER (ORDER BY score_performance) AS quartile,
  CASE NTILE(4) OVER (ORDER BY score_performance)
    WHEN 1 THEN 'Performance faible'
    WHEN 2 THEN 'Performance moyenne-basse'
    WHEN 3 THEN 'Performance moyenne-haute'
    WHEN 4 THEN 'Performance élevée'
  END AS categorie_performance
FROM performances_employes
ORDER BY score_performance;
```

Résumé des fonctions de fenêtrage

Catégorie	Fonction	Description	Cas d'usage
Ranking	ROW_NUMBER()	Attribue un numéro unique à chaque ligne	Numérotation, pagination
	RANK()	Attribue le même rang aux ex-aequo avec sauts	Classements sportifs
	DENSE_RANK()	Attribue le même rang aux ex-aequo sans sauts	Classements scolaires
	NTILE(n)	Divise les lignes en n groupes égaux	Quartiles, déciles
	PERCENT_RANK())	Calcule le rang relatif entre 0 et 1	Analyse de distribution

Agrégation	<code>SUM()</code> <code>OVER()</code>	Calcule la somme sur une fenêtre	Totaux, sous-totaux
	<code>AVG()</code> <code>OVER()</code>	Calcule la moyenne sur une fenêtre	Moyennes glissantes
	<code>COUNT()</code> <code>OVER()</code>	Compte les lignes dans une fenêtre	Densité de distribution
	<code>MIN()/MAX()</code> <code>OVER()</code>	Trouve le min/max dans une fenêtre	Valeurs extrêmes

Navigation`LAG(col, n)`

Accède à la valeur n lignes avant

Comparaison avec période précédente

`LEAD(col, n)`

Accède à la valeur n lignes après

Prévision, tendance

`FIRST_VALUE(c
ol)`

Première valeur de la fenêtre

Référence initiale,
benchmark`LAST_VALUE(co
l)`

Dernière valeur de la fenêtre

Valeur finale, résultat

